Univerzitet u Sarajevu Elektrotehnički fakultet

Predmet: Multimedijalni Sistemi

Laboratorijska vježba 05: Fourierovi redovi

Za izradu laboratorijske vježbe treba koristiti odgovarajuću Jupyter Notebook datoteku. Urađenu vježbu je potrebno konvertirati u PDF format, a zatim je PDF datoteku potrebno predati do postavljenog roka koristeći platformu Zamger.

Ime i prezime studenta, broj indeksa:

Ismar Muslić

Datum izrade izvještaja:

16.04.2026.

Zadatak 1.

Potrebno je implementirati funkciju fourier koja izračunava Fourierove koeficijente a0, a*i* i b*i*, *i*=1, 2, 3..., za funkciju proslijeđenu kao parametar. Osim funkcije, kao parametar se prosljeđuje i period funkcije, kao i broj članova u redu (što zapravo određuje preciznost rekonstrukcije originalne funkcije).

Vrijednosti a0, a*i* i b*i* se izračunavaju prema sljedećim formulama:

$$a_0 = \frac{1}{T}\int_0^T f(x)dx$$

$$a_i = \frac{1}{0.5T}\int_0^T f(x)\cos(ix)\,dx$$

$$b_i = \frac{1}{0.5T}\int_0^T f(x)\sin(ix)\,dx$$

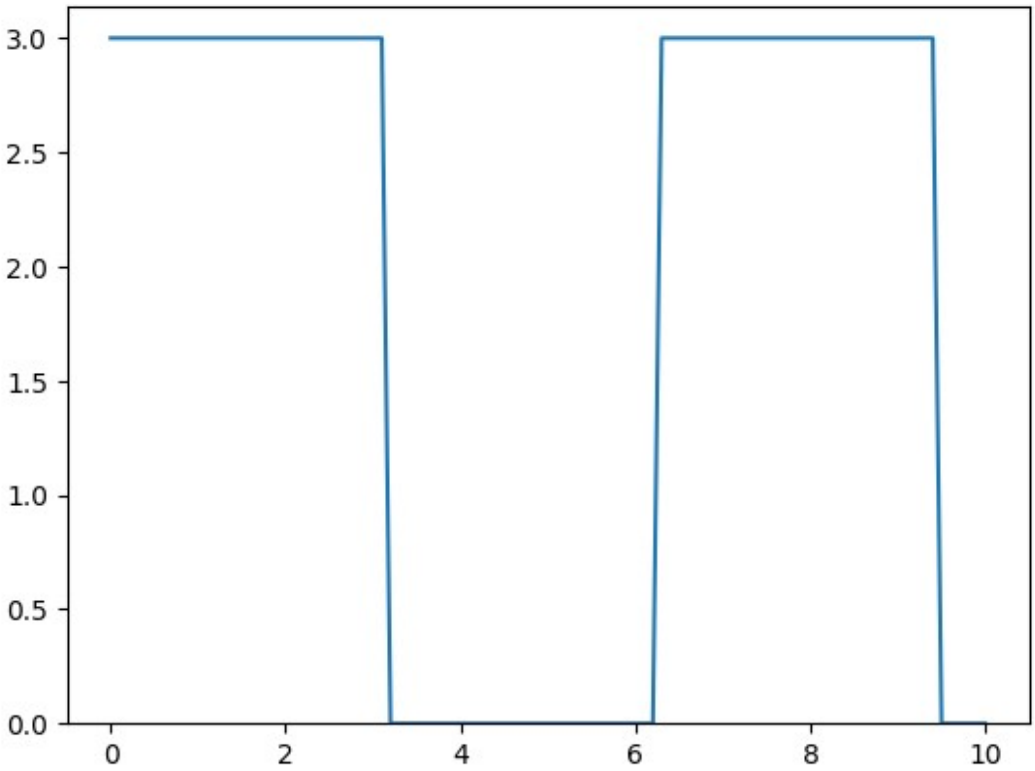
Nije potrebno ručno izračunavati vrijednosti integrala a zatim te vrijednosti koristiti u formulama. Za integriranje možete koristiti funkcija quad iz biblioteke scipy. Ova funkcija kao parametar prima željenu funkciju koju je potrebno integrirati, donju granicu integrala i gornju granicu integrala. Kao rezultat vraća dvije vrijednosti: vrijednost integrala i procjenu apsolutne greške u rezultatu integracije.

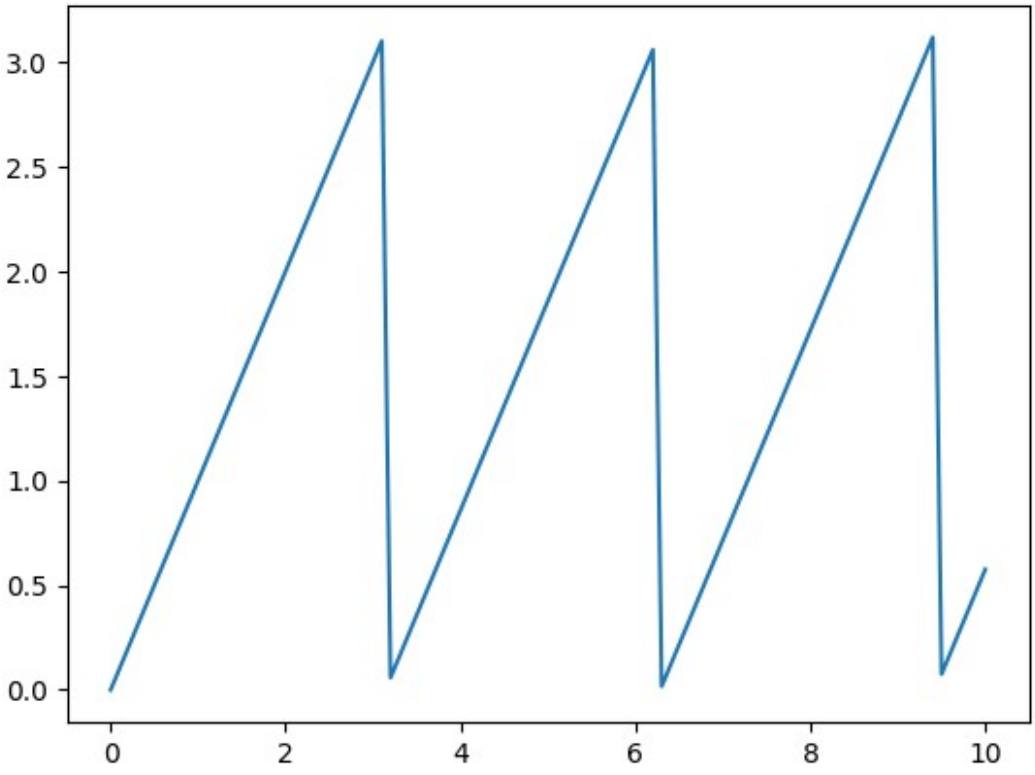
Detaljnije informacije o Fourierovim redovima moguće je pronaći u PDF materijalima u poglavlju 3 (str. 59).

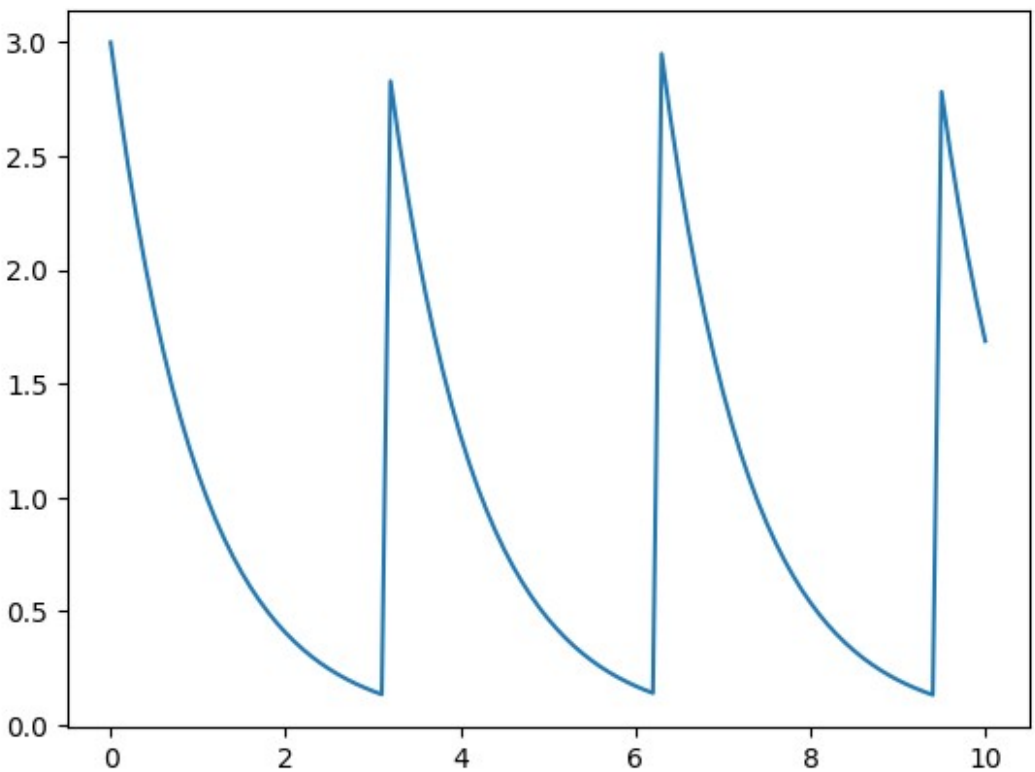
U nastavku su data definicije periodičnih funkcija square, triangle i impulse za koje je potrebno izračunati Fourierove koeficijente. U nastavku su također dati i grafički prikazi navedenih funkcija. Sve navedene funkcije imaju period T=2π.

```
import numpy as np
import matplotlib.pyplot as plt
from math import exp

def square(x, h = 3):
    # Iz definicije funkcije se vidi da je period 2*pi
    ugao = x % (2 * np.pi)
    if 0.0 <= ugao < np.pi:
        return h
    else:
        return 0

def triangle(x):
  # Iz definicije funkcije se vidi da je period pi
  return x % (np.pi)

def impulse(x, h=3):
  # Iz definicije funkcije se vidi da je period pi
  x = x % (np.pi)
  return 3*exp(-x) % (np.pi)



x = [0.0]

f1 = []
f2 = []
f3 = []

for i in range(0, 100):
    f1.append(square(x[len(x) - 1]))
    f2.append(triangle(x[len(x) - 1]))
    f3.append(impulse(x[len(x) - 1]))
    x.append(x[len(x) - 1] + 0.1)
f1.append(square(x[len(x) - 1]))
f2.append(triangle(x[len(x) - 1]))
f3.append(impulse(x[len(x) - 1]))

plt.ylim(0, np.pi)
plt.figure(1)
plt.plot(x, f1)
plt.figure(2)
plt.plot(x, f2)
plt.figure(3)
plt.plot(x, f3)
```

```
[<matplotlib.lines.Line2D at 0x2347fa156a0>]
```







Rješenje:

```
from scipy.integrate import quad

# ***************************!!! VAŽNA NAPOMENA !!!
****************************
#
# Korišten je oblik a_n = 2/T * integral(0, T,
f(t)*cos(2*pi*n*t/T)dt),
#                    b_n = 2/T * integral(0, T, f(t)*sin(2*pi*n*t/T)dt)
i
#                    a_0 = 1/T * integral(0, T, f(t)dt)
#
# Oblik iz postavke zadaće a_n = 2/T * integral(0, T, f(t)*cos(n*t)dt)
i
#                          b_n = 2/T * integral(0, T, f(t)*sin(n*t)dt)
#
# nije ispravan jer nisu za sve tri funkcije periodi 2*pi.
# Ako su trebali biti periodu 2*pi kao što je navedeno u tekstu
postavke,
# kod to ne reflektira. Alternativno je bilo moguće i vratiti funkcije

# da imaju period 2*pi, ali odlučio sam ovako raditi kako su već
postavljene.
```

```
def izracunaj_a_n(f, n, T):
    # Definiramo podintegralnu funkciju kao x(t)*cos(2*pi * n * f * t)
[gdje je f = 1/T]
    def integrand(t):
        return f(t) * np.cos(2 * np.pi * n * 1/T * t)

    # integral(0, T, integrand)
    result, _ = quad(integrand, 0, T, limit=1000)
    return 2 * result / T

def izracunaj_b_n(f, n, T):
    # Definiramo podintegralnu funkciju
    def integrand(t):
        return f(t) * np.sin(2 * np.pi * n * 1/T * t)

    # integral(0, T, integrand)
    result, _ = quad(integrand, 0, T, limit=1000)
    return 2 * result / T

def izracunaj_a_0(f, T):
    # integral(0, T, f(t)dt)
    result, _ = quad(f, 0, T, limit=1000)
    return result / T


def fourier(f, T, n):
    a_n = []
    b_n = []

    a_0 = izracunaj_a_0(f, T)

    for i in range(1, n + 1):
        a_n.append(izracunaj_a_n(f, i, T))
        b_n.append(izracunaj_b_n(f, i, T))
    return a_0, a_n, b_n
```

Nakon implementacije funkcije fourier, treba biti moguće izvršiti programski kod ispod tako da daje prikazani ispis. Vrijednosti a01, a02 i a03 predstavljaju koeficijent a0 za funkcije sqare, triangle i impulse, respektivno. U listama an1, bn1 se trebaju nalaziti koeficijenti a*i* i b*i* za funkciju square. Nadalje, u listama an2, bn2 se trebaju nalaziti koeficijenti a*i* i b*i* za funkciju triangle, dok se u listama an3 i bn3 trebaju nalaziti koeficijenti a*i* i b*i* za funkciju impulse.

```
[a01, an1, bn1] = fourier(square, 2*np.pi, 50)

print("Koeficijent a01 iznosi " + str(round(a01, 2)))

print("\nKoeficijenti ai1 su: ")
an1_string = ""
for i in range(0, len(an1)):
    an1_string += str(round(an1[i], 2)) + " "
```

```
print(an1_string)

print("\nKoeficijenti bi1 su: ")
bn1_string = ""
for i in range(0, len(bn1)):
    bn1_string += str(round(bn1[i], 2)) + " "

print(bn1_string)

# Greška: period je pi a ne 2*pi, pa su koeficijenti pogrešni.
#[a02, an2, bn2] = fourier(triangle, 2 * np.pi, 50)

[a02, an2, bn2] = fourier(triangle, np.pi, 50)

print("\n\nKoeficijent a02 iznosi " + str(round(a02, 2)))

print("\nKoeficijenti ai2 su: ")
an2_string = ""
for i in range(0, len(an2)):
    an2_string += str(round(an2[i], 2)) + " "

print(an2_string)

print("\nKoeficijenti bi2 su: ")
bn2_string = ""
for i in range(0, len(bn2)):
    bn2_string += str(round(bn2[i], 2)) + " "

print(bn2_string)

# Greška: period je pi a ne 2*pi, pa su koeficijenti pogrešni.
#[a03, an3, bn3] = fourier(impulse, 2 * np.pi, 50)
[a03, an3, bn3] = fourier(impulse, np.pi, 50)

print("\n\nKoeficijent a03 iznosi " + str(round(a03, 2)))

print("\nKoeficijenti ai3 su: ")
an3_string = ""
for i in range(0, len(an3)):
    an3_string += str(round(an2[i], 2)) + " "

print(an3_string)

print("\nKoeficijenti bi3 su: ")
bn3_string = ""
for i in range(0, len(bn3)):
    bn3_string += str(round(bn3[i], 2)) + " "

print(bn3_string)
```

```
Koeficijent a01 iznosi 1.5

Koeficijenti ai1 su:
0.0 0.0 -0.0 -0.0 0.0 -0.0 -0.0 -0.0 0.0 -0.0 0.0 -0.0 0.0 -0.0 0.0 -
0.0 -0.0 0.0 -0.0 -0.0 0.0 -0.0 -0.0 -0.0 -0.0 0.0 -0.0 -0.0 0.0 0.0 -
0.0 0.0 0.0 -0.0 0.0 -0.0 -0.0 0.0 0.0 -0.0 -0.0 0.0 -0.0 0.0 -0.0 0.0
-0.0 -0.0 0.0 -0.0

Koeficijenti bi1 su:
1.91 0.0 0.64 -0.0 0.38 0.0 0.27 -0.0 0.21 -0.0 0.17 0.0 0.15 -0.0
0.13 0.0 0.11 -0.0 0.1 -0.0 0.09 0.0 0.08 -0.0 0.08 -0.0 0.07 -0.0
0.07 0.0 0.06 -0.0 0.06 0.0 0.05 -0.0 0.05 -0.0 0.05 0.0 0.05 -0.0
0.04 0.0 0.04 -0.0 0.04 -0.0 0.04 0.0


Koeficijent a02 iznosi 1.57

Koeficijenti ai2 su:
0.0 -0.0 0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 0.0 -0.0 -
0.0 0.0 0.0 0.0 -0.0 0.0 0.0 -0.0 0.0 -0.0 0.0 0.0 -0.0 -0.0 -0.0 -0.0
-0.0 0.0 0.0 -0.0 -0.0 -0.0 -0.0 0.0 -0.0 -0.0 0.0 -0.0 -0.0 0.0 0.0 -
0.0 0.0 -0.0 0.0 -0.0

Koeficijenti bi2 su:
-1.0 -0.5 -0.33 -0.25 -0.2 -0.17 -0.14 -0.12 -0.11 -0.1 -0.09 -0.08 -
0.08 -0.07 -0.07 -0.06 -0.06 -0.06 -0.05 -0.05 -0.05 -0.05 -0.04 -0.04
-0.04 -0.04 -0.04 -0.04 -0.03 -0.03 -0.03 -0.03 -0.03 -0.03 -0.03 -
0.03 -0.03 -0.03 -0.03 -0.02 -0.02 -0.02 -0.02 -0.02 -0.02 -0.02 -0.02
-0.02 -0.02 -0.02


Koeficijent a03 iznosi 0.91

Koeficijenti ai3 su:
0.0 -0.0 0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 0.0 -0.0 -
0.0 0.0 0.0 0.0 -0.0 0.0 0.0 -0.0 0.0 -0.0 0.0 0.0 -0.0 -0.0 -0.0 -0.0
-0.0 0.0 0.0 -0.0 -0.0 -0.0 -0.0 0.0 -0.0 -0.0 0.0 -0.0 -0.0 0.0 0.0 -
0.0 0.0 -0.0 0.0 -0.0

Koeficijenti bi3 su:
0.73 0.43 0.3 0.22 0.18 0.15 0.13 0.11 0.1 0.09 0.08 0.08 0.07 0.07
0.06 0.06 0.05 0.05 0.05 0.05 0.04 0.04 0.04 0.04 0.04 0.04 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.02 0.02 0.02 0.02 0.02 0.02
0.02 0.02 0.02 0.02 0.02 0.02 0.02 0.02
```

Zadatak 2.

Potrebno je implementirati funkciju f_rekonstrukcija koja rekonstruira vrijednosti funkcije na nekom segmentu na temelju Fourierovih koeficijenata koji su izračunati u prethodnom zadatku.

Za rekonstrukciju funkcije u nekoj tački x koristi se sljedeća formula:

$$f(x) = a_0 + \sum_{i=1}^{n} a_i \cos(ix) + \sum_{i=1}^{n} b_i \sin(ix)$$

Rekonstrukcija se izvodi u tačkama x koje su definirane parametrima: početak segmenta, kraj segmenta i step. Naprimjer, za početak segmenta 0, kraj segmenta 0.5 i step 0.1, vrijednosti x za koje se rekonstruira funkcija su: 0.0, 0.1, 0.2, 0.3, 0.4 i 0.5.

Funkcija f_rekonstrukcija kao parametre prima: k (broj komponenti, tj. harmonika na temelju kojih se izvodi rekonstrukcija), fun (funkcija koja se rekonstruira), prethodno izračunate koeficijente a0, an i bn, start (početak segmenta), end (kraj segmenta), step (korak diskretizacije).

Funkcija f_rekonstrukcija kao rezultat vraća liste x i y. U listi x su pohranjene sve vrijednosti x, dok su u listi y pohranjene rekonstruirane vrijednosti funkcije.

Rješenje:

```
def f_rekonstrukcija(k, fun, a0, an, bn, start, end, step):
    x = []
    f = []
    T = 0

    # Odredimo period funkcije fun
    if fun == square:
        T = 2*np.pi
    elif fun == triangle or fun == impulse:
        T = np.pi

    # Suma od 1 do k, gdje je k broj članova Fourierovog reda koji
koristimo za rekonstrukciju funkcije
    for i in np.arange(start, end, step):
        # Tačka u kojoj se gleda f(t)
        x.append(i)

        # Početna vrijednost funkcije f(t) je a0
        f.append(a0)

        # Dodajemo članove Fourierovog reda po formuli f(t) = a0 +
sum(an * cos(2*pi*n*t/T) + bn * sin(2*pi*n*t/T))
        for n in range(1, k + 1):
            f[len(f) - 1] += an[n - 1] * np.cos(2 * np.pi * n *
x[len(x) - 1] / T) + bn[n - 1] * np.sin(2 * np.pi * n * x[len(x) - 1]
/ T)
```

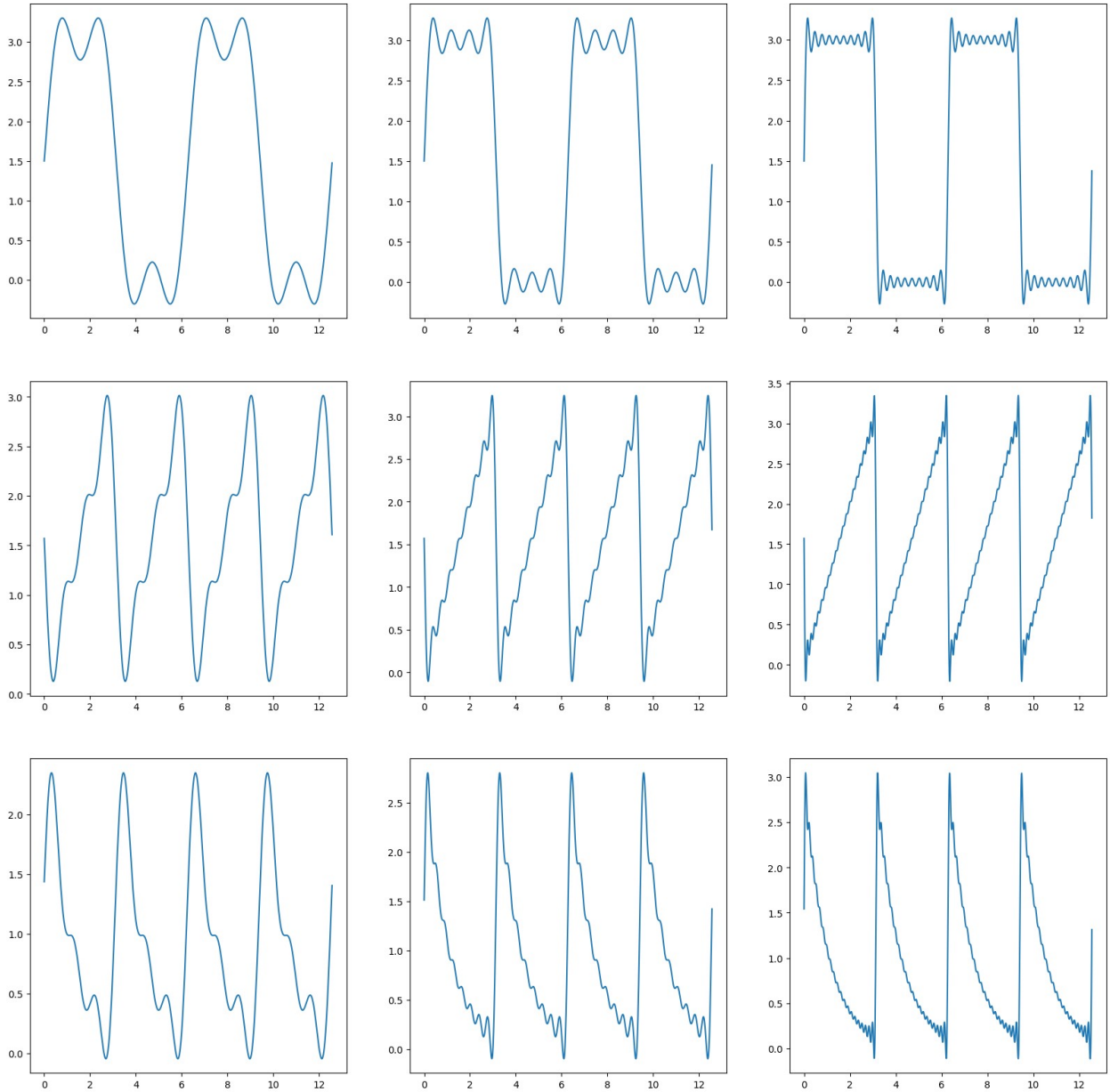
```
    return x, f
```

Nakon implementacije funkcije, potrebno je biti moguće izvršiti programski kod ispod tako da se dobije prikazani grafički prikaz rekonstruiranih funkcija. U primjerima ispod je primijenjena rekonstrukcija korištenjem, 3, 8 i 20 komponenti (harmonika). Za testiranje dodajte još primjere koji koriste 16 i 50 komponenti.

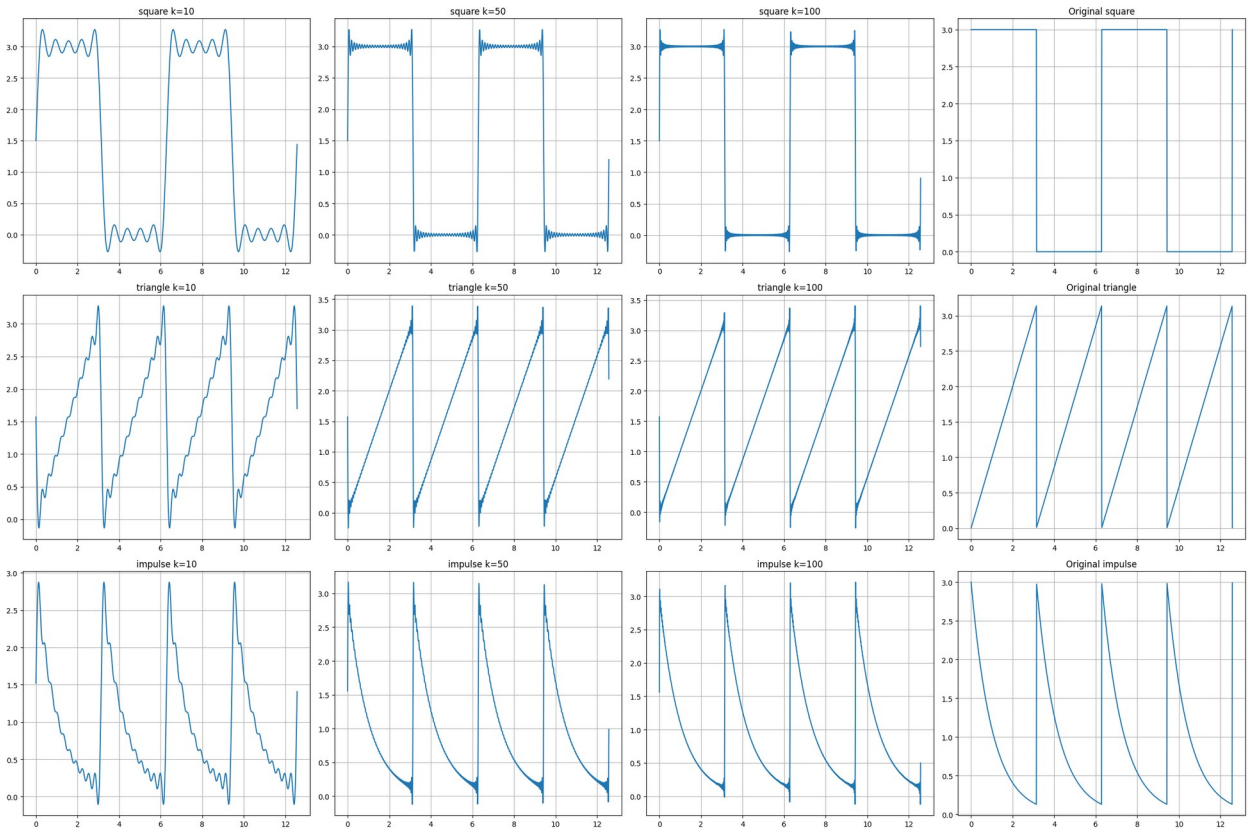
```
# U pozivima se traži da se iscrta funkcija sa 3, 8 i 20 članova, ali
slike
# funkcija koje su stavljene u postavku imaju 1, 4, i 10 članova za
# triangle i impulse

# Razlog je što zbog pogrešno postavljenog perioda gore ima upola
manje
# koeficijenata, pa je 3, 8 i 20 zapravo 1, 4 i 10.

[x, rf1_3] = f_rekonstrukcija(3,square, a01, an1, bn1, 0, 4 * np.pi,
0.01)
[x, rf1_8] = f_rekonstrukcija(8,square, a01, an1, bn1, 0, 4 * np.pi,
0.01)
[x, rf1_20] = f_rekonstrukcija(20,square, a01, an1, bn1, 0, 4 * np.pi,
0.01)

[x, rf2_3] = f_rekonstrukcija(3,triangle, a02, an2, bn2, 0, 4 * np.pi,
0.01)
[x, rf2_8] = f_rekonstrukcija(8,triangle, a02, an2, bn2, 0, 4 * np.pi,
0.01)
[x, rf2_20] = f_rekonstrukcija(20,triangle, a02, an2, bn2, 0, 4 *
np.pi, 0.01)

[x, rf3_3] = f_rekonstrukcija(3,impulse, a03, an3, bn3, 0, 4 * np.pi,
0.01)
[x, rf3_8] = f_rekonstrukcija(8,impulse, a03, an3, bn3, 0, 4 * np.pi,
0.01)
[x, rf3_20] = f_rekonstrukcija(20,impulse, a03, an3, bn3, 0, 4 *
np.pi, 0.01)


fig, axs = plt.subplots(3,3,figsize=(20,20))

axs[0,0].plot(x,rf1_3)
axs[0,1].plot(x,rf1_8)
axs[0,2].plot(x,rf1_20)

axs[1,0].plot(x,rf2_3)
axs[1,1].plot(x,rf2_8)
axs[1,2].plot(x,rf2_20)

axs[2,0].plot(x,rf3_3)
```

```
axs[2,1].plot(x,rf3_8)
axs[2,2].plot(x,rf3_20)
```

```
[<matplotlib.lines.Line2D at 0x2347ea4e270>]
```



```
[a01, an1, bn1] = fourier(square, 2*np.pi, 100)
[a02, an2, bn2] = fourier(triangle, np.pi, 100)
[a03, an3, bn3] = fourier(impulse, np.pi, 100)

[x, rf1_3] = f_rekonstrukcija(10,square, a01, an1, bn1, 0, 4 * np.pi,
0.01)
[x, rf1_20] = f_rekonstrukcija(50,square, a01, an1, bn1, 0, 4 * np.pi,
```

```
0.01)
[x, rf1_100] = f_rekonstrukcija(100,square, a01, an1, bn1, 0, 4 *
np.pi, 0.01)

[x, rf2_3] = f_rekonstrukcija(10,triangle, a02, an2, bn2, 0, 4 *
np.pi, 0.01)
[x, rf2_20] = f_rekonstrukcija(50,triangle, a02, an2, bn2, 0, 4 *
np.pi, 0.01)
[x, rf2_100] = f_rekonstrukcija(100,triangle, a02, an2, bn2, 0, 4 *
np.pi, 0.01)

[x, rf3_3] = f_rekonstrukcija(10,impulse, a03, an3, bn3, 0, 4 * np.pi,
0.01)
[x, rf3_20] = f_rekonstrukcija(50,impulse, a03, an3, bn3, 0, 4 *
np.pi, 0.01)
[x, rf3_100] = f_rekonstrukcija(100,impulse, a03, an3, bn3, 0, 4 *
np.pi, 0.01)

x_orig = np.arange(0, 4 * np.pi + 0.01, 0.01)

fig, axs = plt.subplots(3, 4, figsize=(24, 16))


axs[0, 0].plot(x, rf1_3)
axs[0, 0].set_title("square k=10")
axs[0, 1].plot(x, rf1_20)
axs[0, 1].set_title("square k=50")
axs[0, 2].plot(x, rf1_100)
axs[0, 2].set_title("square k=100")
axs[0, 3].plot(x_orig, [square(xi) for xi in x_orig])
axs[0, 3].set_title("Original square")


axs[1, 0].plot(x, rf2_3)
axs[1, 0].set_title("triangle k=10")
axs[1, 1].plot(x, rf2_20)
axs[1, 1].set_title("triangle k=50")
axs[1, 2].plot(x, rf2_100)
axs[1, 2].set_title("triangle k=100")
axs[1, 3].plot(x_orig, [triangle(xi) for xi in x_orig])
axs[1, 3].set_title("Original triangle")


axs[2, 0].plot(x, rf3_3)
axs[2, 0].set_title("impulse k=10")
axs[2, 1].plot(x, rf3_20)
axs[2, 1].set_title("impulse k=50")
axs[2, 2].plot(x, rf3_100)
axs[2, 2].set_title("impulse k=100")
axs[2, 3].plot(x_orig, [impulse(xi) for xi in x_orig])
```

```python
axs[2, 3].set_title("Original impulse")

for ax in axs.flatten():
    ax.grid(True)

plt.tight_layout()
```